Interactive Neural Network Learning Platform

---

1. Introduction

This project presents an interactive educational platform designed to help users understand how neural networks work through experimentation and visualization. Unlike traditional implementations that focus solely on performance, this platform emphasizes interpretability, allowing users to observe how different models learn and generalize.

The platform enables users to:

- Load datasets
- Configure models and hyperparameters
- Train neural networks
- Visualize learning dynamics
- Compare model performance

---

2. Objectives

The main objectives of this project are:

- To implement fundamental and advanced neural network models from scratch
- To provide an interactive environment for experimentation
- To visualize the learning process in real time
- To analyze model performance and generalization

---

3. System Architecture

The platform is composed of three main components:

3.1 Backend (Python)

The backend includes:

- **Models**
    - Perceptron
    - Multi-Layer Perceptron (MLP)

- **Training**
  - Forward propagation
  - Backpropagation
  - Gradient descent optimization
- **Utilities**
  - Activation functions (ReLU, Sigmoid, Tanh, Softmax)
  - Loss functions (MSE, Binary Cross-Entropy, Categorical Cross-Entropy)
  - Evaluation metrics

3.2 Data Pipeline

The system includes:

- Data cleaning (handling missing values)
- Feature selection (numeric filtering)
- Train/test split
- Optional standardization

3.3 Frontend (Interactive UI)

The platform is implemented as an **interactive notebook interface using widgets**, allowing users to:

- Select target variables
- Choose model type
- Configure architecture and hyperparameters
- Train models with a single button
- Visualize results dynamically

This approach provides a lightweight yet fully interactive user interface suitable for educational purposes.

---

4. Theoretical Background

Neural Networks

Neural networks are composed of layers of interconnected neurons. Each neuron performs a linear transformation followed by a non-linear activation function.

Forward Propagation

Forward propagation computes predictions by passing input data through the network layer by layer.

Backpropagation

Backpropagation computes gradients of the loss function with respect to model parameters using the chain rule.

Gradient Descent

Gradient descent updates model parameters iteratively to minimize the loss function.

Bias vs Variance

- **Bias** → underfitting
- **Variance** → overfitting

A good model balances both.

---

5. Experiments

5.1 Experiment 1 — Perceptron vs MLP

The perceptron produces a linear decision boundary and fails to classify non-linear data effectively. The MLP, with hidden layers and non-linear activation functions, successfully captures complex pattern

Figure 1: Decision boundary learned by the perceptron, showing its limitation to linear separation.

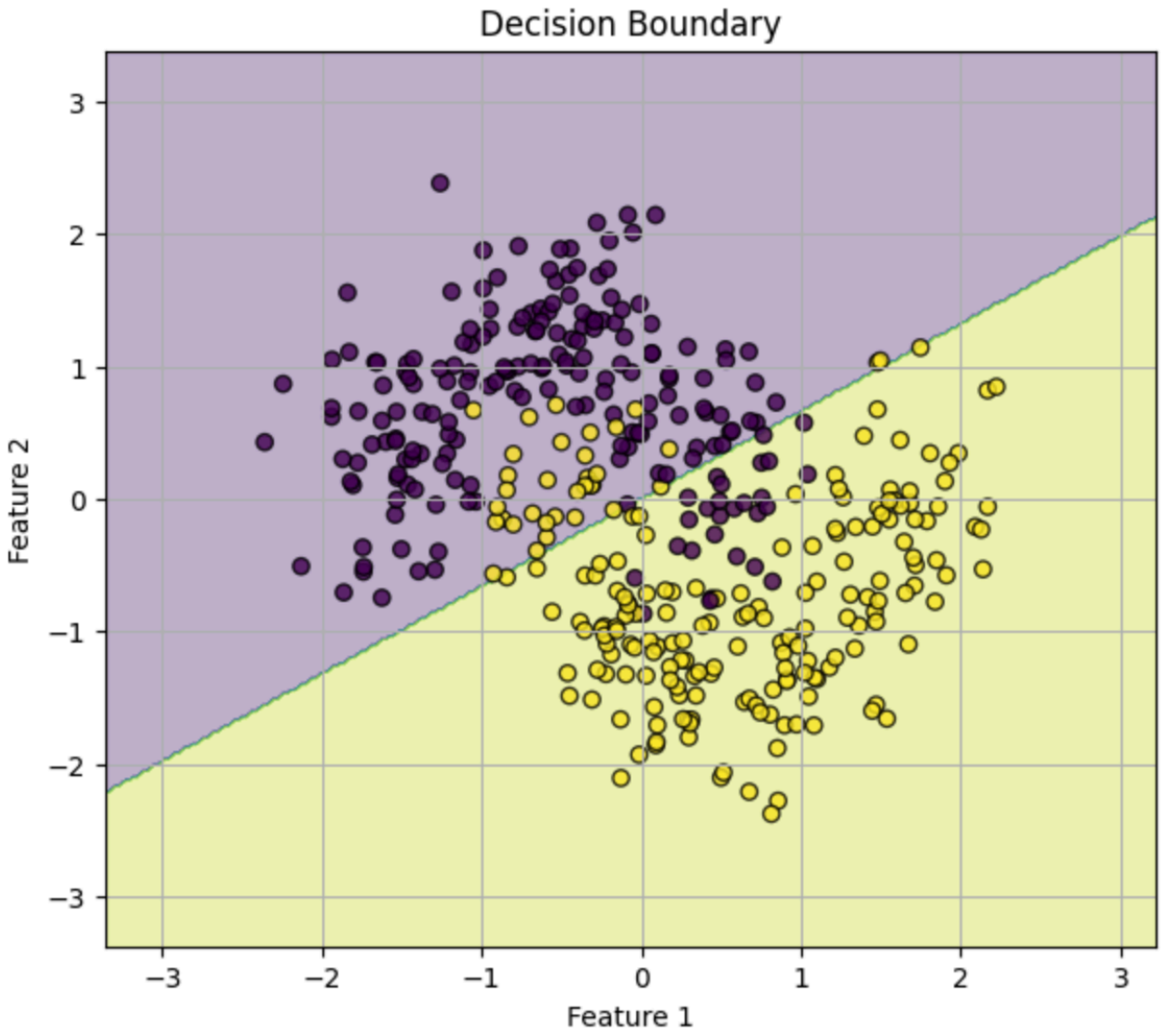
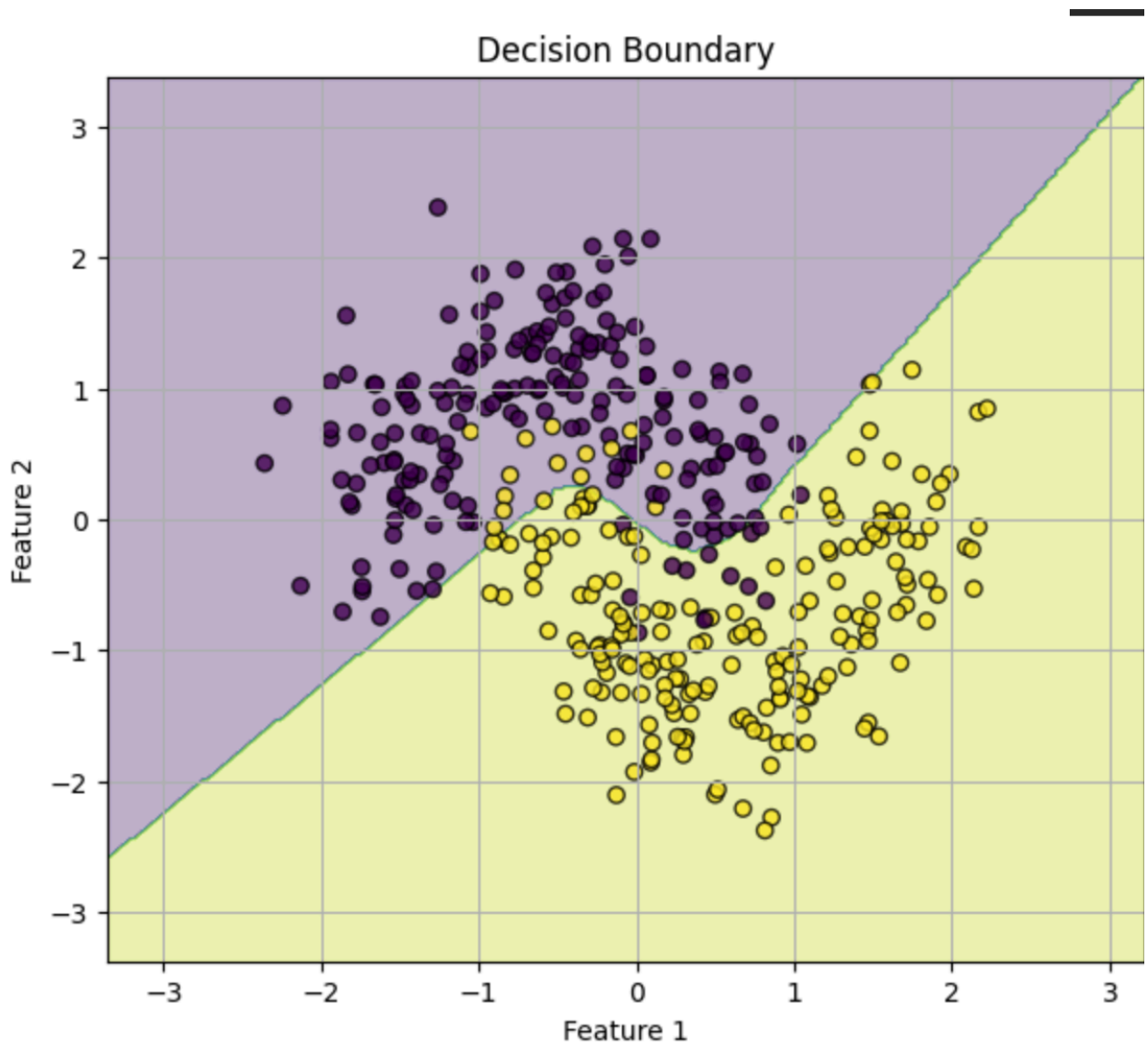


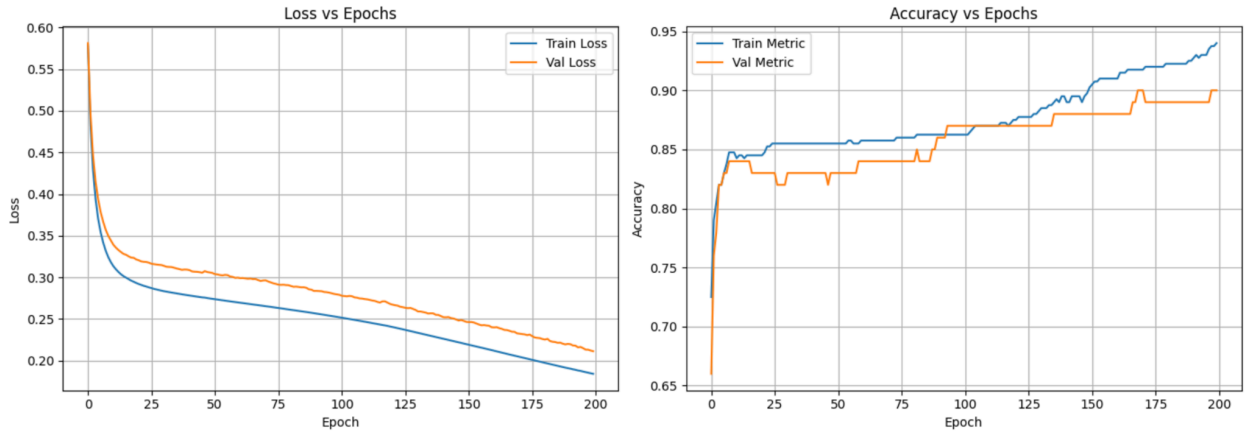
Figure 2: Decision boundary learned by the MLP, demonstrating non-linear learning capability.



5.2 Experiment 2 — Effect of Network Depth

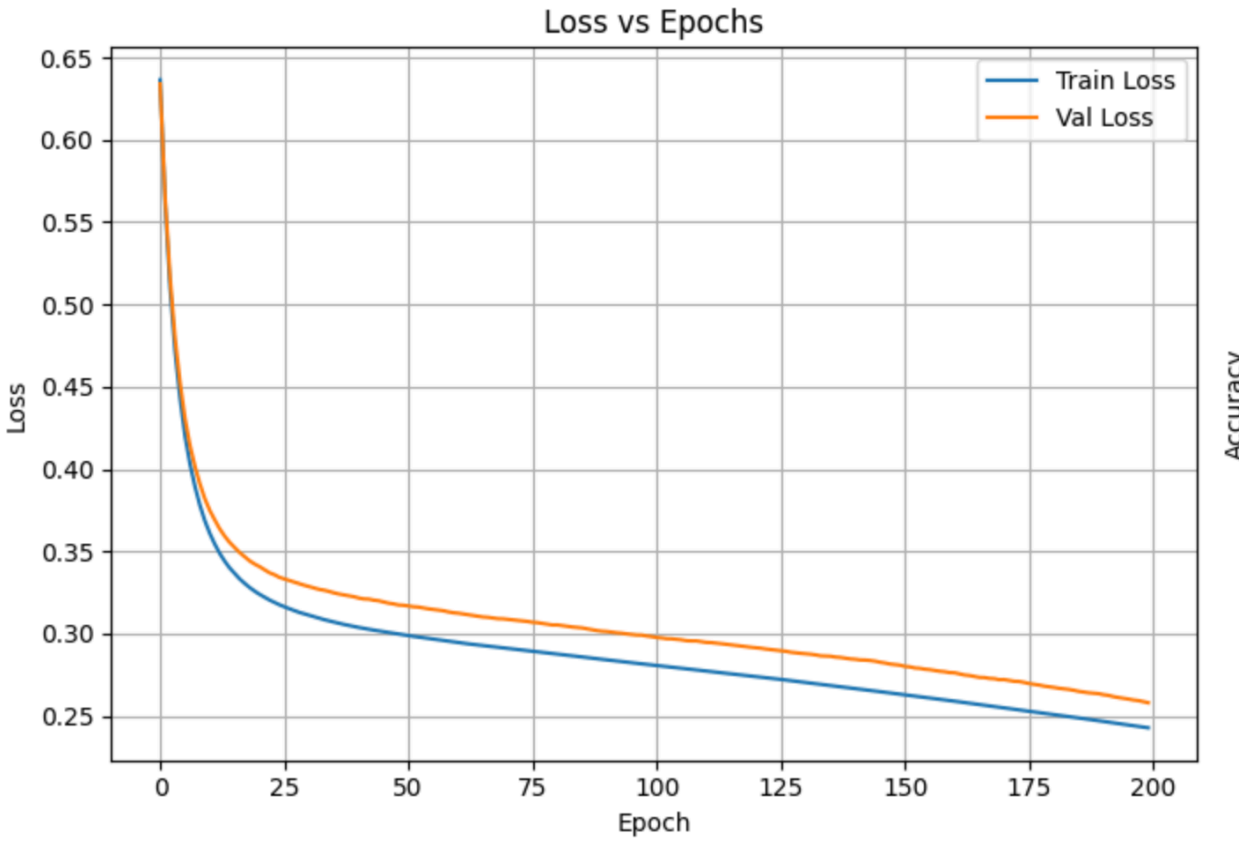
Increasing the number of layers improves the model's ability to learn complex relationships. However, deeper networks may overfit if not properly regularized.

Figure 3: Effect of network depth on model performance.



5.3 Experiment 3 — Regularization

Without regularization, the model overfits the training data. With L2 regularization and early stopping, the model generalizes better to unseen data.
Figure 4: Training behavior illustrating overfitting and the impact of regularization.



---

6. Visualization

The platform provides:

- Loss vs Epochs
- Accuracy vs Epochs
- Decision boundary (2D)
- Confusion matrix

These visualizations allow users to understand how models learn over time.

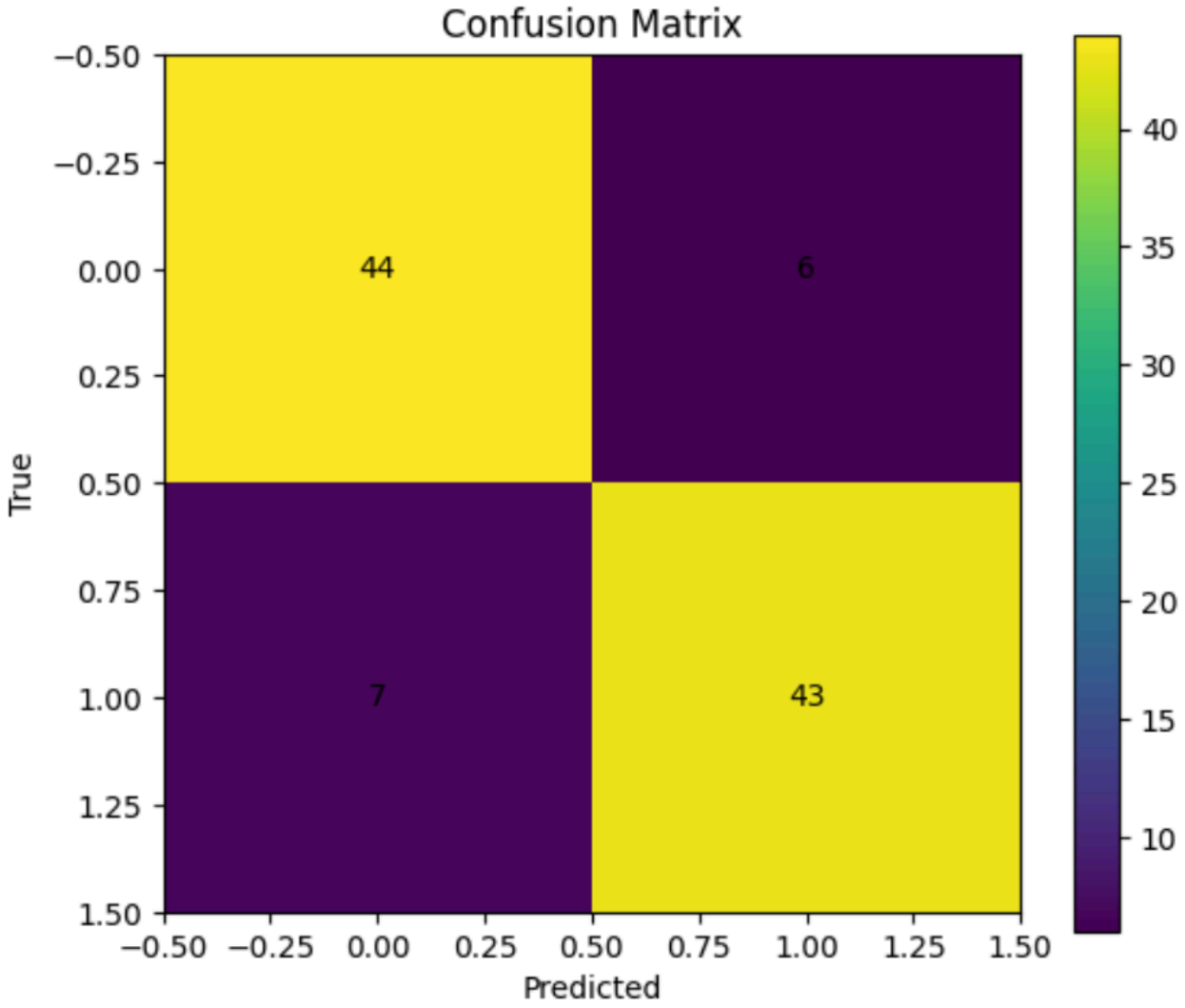
---

7. Model Evaluation

Classification performance is evaluated using:

- Accuracy
- Precision
- Recall
- F1-score

- Confusion matrix
  Figure 5: Confusion matrix showing classification performance.



---

8. Bonus Features

sklearn Comparison

The custom implementation was compared with sklearn models. The results were similar, validating correctness while providing deeper insight into the learning process.

Figure 6: Comparison of model configurations and performance metrics.

	target_col	task_type	model_type	hidden_layers	hidden_activation	learning_rate	epochs	batch_size	l2_lambda	patience	test_size	standardize	early_stopping	verbose	sklearn_compare	accuracy	precision	recall	f1
0	target	binary	mlp	[16, 16]	relu	0.01	200	32	0.001	20	0.2	True	True	True	True	0.86	0.821429	0.92	0.867925
1	target	binary	mlp	[16, 16]	relu	0.01	200	32	0.001	20	0.2	True	True	True	True	0.86	0.846154	0.88	0.862745
2	target	binary	mlp	[4]	relu	0.01	200	32	0.001	20	0.2	True	True	True	True	0.84	0.793103	0.92	0.851852
3	target	binary	mlp	[4]	relu	0.01	200	32	0.001	20	0.2	True	True	True	True	0.83	0.823529	0.84	0.831683
4	target	binary	mlp	[16]	relu	0.01	200	32	0.001	20	0.2	True	True	True	True	0.84	0.803571	0.90	0.849057
5	target	binary	mlp	[16, 16]	relu	0.01	200	32	0.001	20	0.2	True	True	True	True	0.89	0.897959	0.88	0.888889
6	target	binary	mlp	[32, 32, 32]	relu	0.01	200	32	0.001	20	0.2	True	True	True	True	0.90	0.884615	0.92	0.901961
7	target	binary	historical_perceptron	[16, 16]	relu	0.01	200	32	0.001	20	0.2	True	True	True	True	0.83	0.800000	0.88	0.838095
8	target	binary	mlp	[16, 16]	relu	0.01	200	32	0.001	20	0.2	True	True	True	True	0.90	0.884615	0.92	0.901961
9	target	binary	mlp	[16, 16]	relu	0.01	200	32	0.000	20	0.2	True	True	True	True	0.86	0.860000	0.86	0.860000
10	target	binary	mlp	[16, 16]	relu	0.01	200	32	0.000	20	0.2	True	False	True	True	0.87	0.877551	0.86	0.868687
11	target	binary	mlp	[16, 16]	relu	0.01	200	32	0.010	20	0.2	True	True	True	True	0.87	0.877551	0.86	0.868687

---

9. Conclusion

This project demonstrates how neural networks can be implemented and understood through an interactive platform. By combining theoretical concepts with experimentation and visualization, the system provides a comprehensive learning tool.